



All pairs shortest paths

Giovedì 21 Maggio 2026

Single source shortest paths: algoritmi a confronto

algoritmo	restrizione	caso peggiore	caso tipico	spazio
topological sort	no cicli	$E + V$	$E + V$	V
Dijkstra (binary heap)	no pesi negativi	$E \log V$	$E \log V$	V
Bellman-Ford	no cicli negativi	EV	EV	V
Bellman-Ford con coda	no cicli negativi	EV	$E + V$	V

All pairs shortest paths: algoritmo di Floyd-Warshall

- Elegante applicazione della tecnica della programmazione dinamica
- Calcola le distanze (peso del cammino minimo o più leggero) fra tutte le coppie di vertici in un grafo orientato con $|V|$ vertici in tempo $O(|V|^3)$.
- I pesi degli archi possono essere negativi, purché non ci siano cicli negativi.

Programmazione dinamica

- Tecnica **bottom-up**:
- 1. Identifica dei sottoproblemi del problema originario, procedendo logicamente dai problemi più piccoli verso quelli più grandi
- 2. Utilizza una **tabella** per memorizzare le soluzioni dei sottoproblemi incontrati: quando si incontra lo stesso sottoproblema, sarà sufficiente esaminare la tabella
- 3. Si usa quando **i sottoproblemi non sono indipendenti**, e lo stesso sottoproblema può apparire più volte

Esempio: numeri di Fibonacci

- Algoritmo Fibonacci(intero n)
F = array di $(n+1)$ interi
F[0]=0
F[1]=1
for i=2 to n do
 F[i]=F[i-1]+F[i-2]
return F[n]
- La tabella viene «programmata dinamicamente»

Distanza tra stringhe

Siano X e Y due stringhe di lunghezza m ed n :

$$X = x_1 \cdot x_2 \cdot \dots \cdot x_m \qquad Y = y_1 \cdot y_2 \cdot \dots \cdot y_n$$

Vogliamo calcolare la “distanza” tra X e Y , ovvero il minimo numero delle seguenti operazioni elementari che permetta di trasformare X in Y

- `inserisci( $a$ ):` Inserisci il carattere a nella posizione corrente della stringa.
- `cancella( $a$ ):` Cancella il carattere a dalla posizione corrente della stringa.
- `sostituisci( $a, b$ ):` Sostituisci il carattere a con il carattere b nella posizione corrente della stringa.

Esempio: distanza tra RISOTTO e PRESTO

Azione	Costo	Stringa ottenuta
Inserisco P	1	P RISOTTO
Mantengo R	0	PR ISOTTO
Sostituisco I con E	1	PRE SOTTO
Mantengo S	0	PRES OTTO
Cancello O	1	PRES TTO
Mantengo T	0	PREST TO
Cancello T	1	PREST O
Mantengo O	0	PRESTO

Approccio

- Denotiamo con $\delta(X, Y)$ la distanza tra X e Y
 - Definiamo X_i il prefisso di X fino all' i -esimo carattere, per $0 \leq i \leq m$ (X_0 denota la stringa vuota):

$$X_i = x_1 \cdot x_2 \cdot \dots \cdot x_i \text{ se } i \geq 1$$

- Risolveremo il problema di calcolare $\delta(X, Y)$ calcolando $\delta(X_i, Y_j)$ per ogni i, j tali che $0 \leq i \leq m$ e $0 \leq j \leq n$

Inizializzazione della tabella

- Alcuni sottoproblemi sono molto semplici
- $\delta(X_0, Y_j)=j$ partendo dalla stringa vuota X_0 , basta inserire uno ad uno j caratteri di Y_j
- $\delta(X_i, Y_0)=i$ partendo da X_i , basta rimuovere uno ad uno gli i caratteri per ottenere la stringa vuota Y_0
- Queste soluzioni sono memorizzate rispettivamente nella prima riga e nella prima colonna della tabella D

Avanzamento nella tabella (1/3)

- Se $x_i = y_j$, il minimo costo per trasformare X_i in Y_j è uguale al minimo costo per trasformare X_{i-1} in Y_{j-1}

$$D[i, j] = D[i-1, j-1]$$

- Se $x_i \neq y_j$, distinguiamo in base all'ultima operazione usata per trasformare X_i in Y_j in una sequenza ottima di operazioni

Avanzamento nella tabella 2/3

`inserisci( $y_j$ )`: il minimo costo per trasformare X_i in Y_j è uguale al minimo costo per trasformare X_i in Y_{j-1} più 1 per inserire il carattere y_j

$$\Rightarrow D[i, j] = 1 + D[i, j-1]$$

`cancella( $x_i$ )`: il minimo costo per trasformare X_i in Y_j è uguale al minimo costo per trasformare X_{i-1} in Y_j più 1 per la cancellazione del carattere x_i

$$\Rightarrow D[i, j] = 1 + D[i-1, j]$$

Avanzamento nella tabella (3/3)

`sostituisci`(x_i, y_j): il minimo costo per trasformare X_i in Y_j è uguale al minimo costo per trasformare X_{i-1} in Y_{j-1} più 1 per sostituire il carattere x_i per y_j

⇒ $D[i, j] = 1 + D[i-1, j-1]$

In conclusione

$$D[i, j] = \begin{cases} D[i-1, j-1] & \text{se } x_i = y_j \\ 1 + \min\{D[i, j-1], D[i-1, j], D[i-1, j-1]\} & \text{se } x_i \neq y_j \end{cases}$$

Pseudocodice

```
algoritmo distanzaStringhe(stringa  $X$ , stringa  $Y$ )  $\rightarrow$  intero  
  matrice  $D$  di  $(m + 1) \times (n + 1)$  interi  
  for  $i = 0$  to  $m$  do  $D[i, 0] \leftarrow i$   
  for  $j = 1$  to  $n$  do  $D[0, j] \leftarrow j$   
  for  $i = 1$  to  $m$  do  
    for  $j = 1$  to  $n$  do  
      if  $(x_i \neq y_j)$  then  
         $D[i, j] \leftarrow 1 + \min\{D[i, j - 1], D[i - 1, j], D[i - 1, j - 1]\}$   
      else  $D[i, j] \leftarrow D[i - 1, j - 1]$   
  return  $D[m, n]$ 
```

Tempo di esecuzione ed occupazione di
memoria: $O(mn)$

Esempio

		P	R	E	S	T	O
	0	1	2	3	4	5	6
R	1	1	1	2	3	4	5
I	2	2	2	2	3	4	5
S	3	3	3	3	2	3	4
O	4	4	4	4	3	3	3
T	5	5	5	5	4	3	4
T	6	6	6	6	5	4	4
O	7	7	7	7	6	5	4

La tabella D costruita dall'algoritmo.

In grassetto sono indicate le sequenze di operazioni che permettono di ottenere la distanza tra le stringhe

Torniamo all'algoritmo di Floyd-Warshall

- Calcola le distanze (peso del cammino minimo o più leggero) fra tutte le coppie di vertici in un grafo orientato con $|V|$ vertici in tempo $O(|V|^3)$.
- I pesi degli archi possono essere negativi, purchè non ci siano cicli negativi.

Cammino minimo k-vincolato

- Sia G un grafo orientato pesato, siano $\{v_1, v_2, \dots, v_n\}$ i suoi vertici.
- Per ogni k compreso tra 0 e n , indichiamo con d_{xy}^k la distanza minima che possiamo percorrere per andare dal vertice x al vertice y senza passare per $v_{k+1}, \dots, v_n$. Questa quantità si chiama distanza k -vincolata.
- Il corrispondente cammino si chiama cammino minimo k -vincolato.

Relazioni tra distanze vincolate

- Sia d_{xy}^k il costo di un cammino minimo k -vincolato da x a y .
Allora:

$$d_{xy}^k = \min\{ d_{xy}^{k-1}, d_{xv_k}^{k-1} + d_{v_k y}^{k-1} \}$$

- Inoltre, $d_{xy}^0 = w(x,y)$ se esiste un arco $x \rightarrow y$, $+\infty$ altrimenti.
- $d_{xy}^0 = 0$ se $x=y$
- $d_{xy}^n = d_{xy}$
- L'algoritmo calcola d_{xy} dal basso verso l'alto, incrementando k da 1 a n .

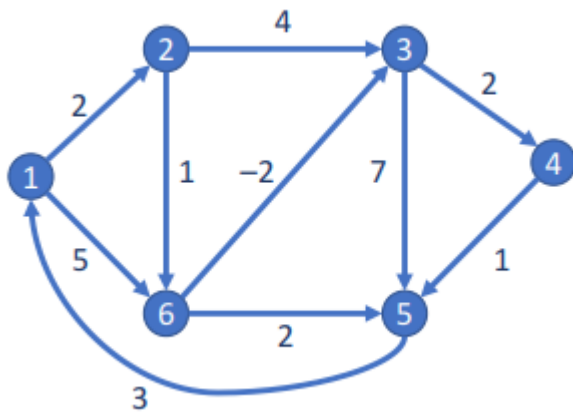
Osservazioni

- Esistono sempre cammini k -vincolati semplici ed essi godono delle proprietà di ottimalità, esattamente come i cammini minimi non vincolati.
- Infatti, basta considerare il grafo G_k in cui non si considerano i vertici $v_{k+1}, \dots, v_n$. Allora un cammino minimo k -vincolato nel grafo di partenza è un cammino minimo semplice nel grafo G_k .

L'algoritmo

- Usa $n+1$ matrici di appoggio $D^0, D^1, \dots, D^n$ di dimensioni $n \times n$ e la tecnica di programmazione dinamica.
- Calcola prima $D^0_{xy} = d^0_{xy}$ per ogni x e y
- Usando la relazione precedentemente descritta, vengono calcolate le matrici D^1 , poiché $D^1_{xy} = d^1_{xy}$ per ogni x e y , e così via fino ad arrivare a $D^n_{xy} = d^n_{xy} = d_{xy}$.
- L'algoritmo richiede un tempo e spazio $O(n^3)$

ESEMPIO



1: $W := (w_{ij})$ con $w_{ij} := \begin{cases} 0 & \text{se } i = j \\ w(i, j) & \text{se } (i, j) \in E(G) \\ \infty & \text{altrimenti} \end{cases}$

2: $D^{(0)} := W$

3: **per** $k := 1, 2, 3, 4, 5, 6$ **ripeti**

4: **per** $i := 1, 2, 3, 4, 5, 6$ **ripeti**

5: **per** $j := 1, 2, 3, 4, 5, 6$ **ripeti**

6: $d_{ij}^{(k)} := \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$

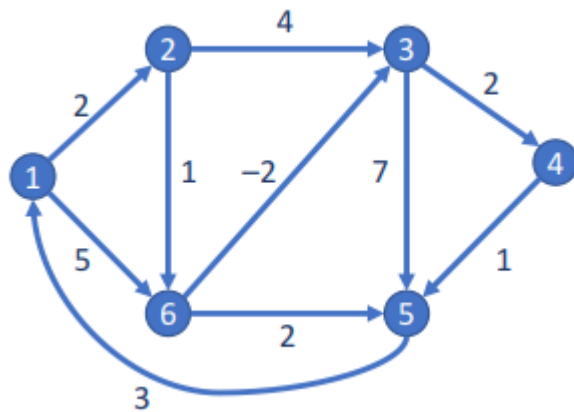
7: **fine-ciclo**

8: **fine-ciclo**

9: **fine-ciclo**

- Viene inizializzata la matrice $D^{(0)}$ con i dati della matrice W

ESEMPIO



$D^{(0)}$

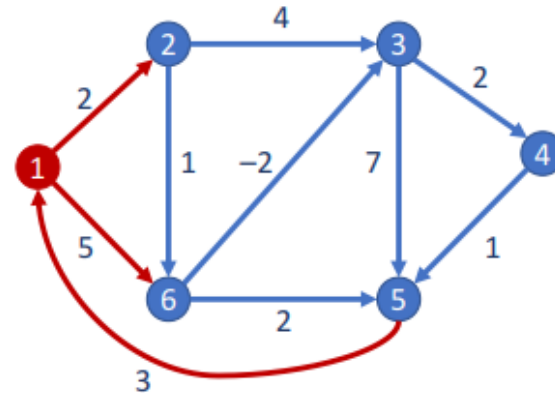
	1	2	3	4	5	6
1	0	2	∞	∞	∞	5
2	∞	0	4	∞	∞	1
3	∞	∞	0	2	7	∞
4	∞	∞	∞	0	1	∞
5	3	∞	∞	∞	0	∞
6	∞	∞	-2	∞	2	0

ESEMPIO

```

1:  $W := (w_{ij})$  con  $w_{ij} := \begin{cases} 0 & \text{se } i = j \\ w(i, j) & \text{se } (i, j) \in E(G) \\ \infty & \text{altrimenti} \end{cases}$ 
2:  $D^{(0)} := W$ 
3: per  $k := 1$  2, 3, 4, 5, 6 ripeti
4:   per  $i := 1, 2, 3, 4, 5, 6$  ripeti
5:     per  $j := 1, 2, 3, 4, 5, 6$  ripeti
6:        $d_{ij}^{(k)} := \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$ 
7:     fine-ciclo
8:   fine-ciclo
9: fine-ciclo
  
```

- Viene calcolata la matrice $D^{(1)}$ con i costi degli eventuali cammini di costo minimo con vertici intermedi in $\{1\}$



$D^{(1)}$

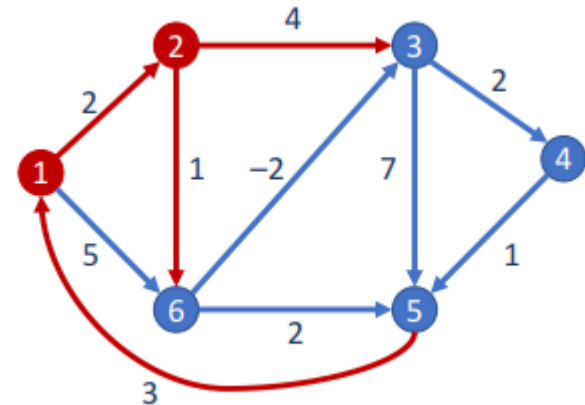
	1	2	3	4	5	6
1	0	2	∞	∞	∞	5
2	∞	0	4	∞	∞	1
3	∞	∞	0	2	7	∞
4	∞	∞	∞	0	1	∞
5	3	5	∞	∞	0	8
6	∞	∞	-2	∞	2	0

ESEMPIO

```

1:  $W := (w_{ij})$  con  $w_{ij} := \begin{cases} 0 & \text{se } i = j \\ w(i, j) & \text{se } (i, j) \in E(G) \\ \infty & \text{altrimenti} \end{cases}$ 
2:  $D^{(0)} := W$ 
3: per  $k := 1, 2, 3, 4, 5, 6$  ripeti
4:   per  $i := 1, 2, 3, 4, 5, 6$  ripeti
5:     per  $j := 1, 2, 3, 4, 5, 6$  ripeti
6:        $d_{ij}^{(k)} := \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$ 
7:     fine-ciclo
8:   fine-ciclo
9: fine-ciclo
  
```

- Viene calcolata la matrice $D^{(2)}$ con i costi degli eventuali cammini di costo minimo con vertici intermedi in $\{1, 2\}$



$D^{(2)}$

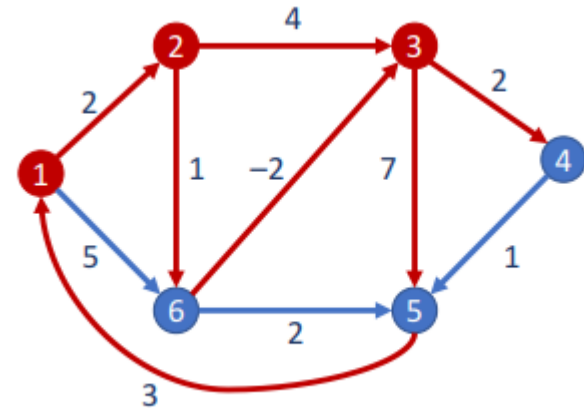
	1	2	3	4	5	6
1	0	2	6	∞	∞	3
2	∞	0	4	∞	∞	1
3	∞	∞	0	2	7	∞
4	∞	∞	∞	0	1	∞
5	3	5	9	∞	0	6
6	∞	∞	-2	∞	2	0

ESEMPIO

```

1:  $W := (w_{ij})$  con  $w_{ij} := \begin{cases} 0 & \text{se } i = j \\ w(i, j) & \text{se } (i, j) \in E(G) \\ \infty & \text{altrimenti} \end{cases}$ 
2:  $D^{(0)} := W$ 
3: per  $k := 1, 2, 3, 4, 5, 6$  ripeti
4:   per  $i := 1, 2, 3, 4, 5, 6$  ripeti
5:     per  $j := 1, 2, 3, 4, 5, 6$  ripeti
6:        $d_{ij}^{(k)} := \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$ 
7:     fine-ciclo
8:   fine-ciclo
9: fine-ciclo
  
```

- Viene calcolata la matrice $D^{(3)}$ con i costi degli eventuali cammini di costo minimo con vertici intermedi in $\{1, 2, 3\}$



$D^{(3)}$

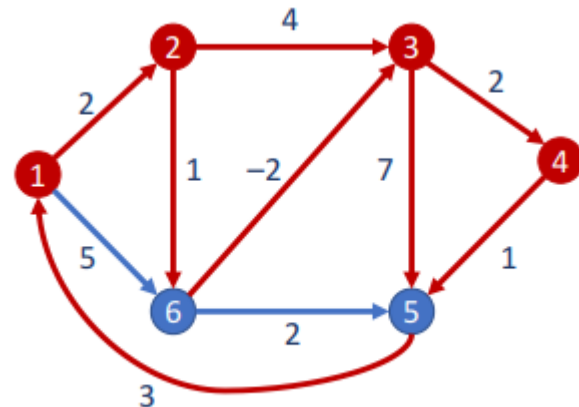
	1	2	3	4	5	6
1	0	2	6	8	13	3
2	∞	0	4	6	11	1
3	∞	∞	0	2	7	∞
4	∞	∞	∞	0	1	∞
5	3	5	9	11	0	6
6	∞	∞	-2	0	2	0

ESEMPIO

```

1:  $W := (w_{ij})$  con  $w_{ij} := \begin{cases} 0 & \text{se } i = j \\ w(i, j) & \text{se } (i, j) \in E(G) \\ \infty & \text{altrimenti} \end{cases}$ 
2:  $D^{(0)} := W$ 
3: per  $k := 1, 2, 3, \boxed{4}, 5, 6$  ripeti
4:   per  $i := 1, 2, 3, 4, 5, 6$  ripeti
5:     per  $j := 1, 2, 3, 4, 5, 6$  ripeti
6:        $d_{ij}^{(k)} := \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$ 
7:     fine-ciclo
8:   fine-ciclo
9: fine-ciclo
  
```

- Viene calcolata la matrice $D^{(4)}$ con i costi degli eventuali cammini di costo minimo con vertici intermedi in $\{1, 2, 3, 4\}$



$D^{(4)}$

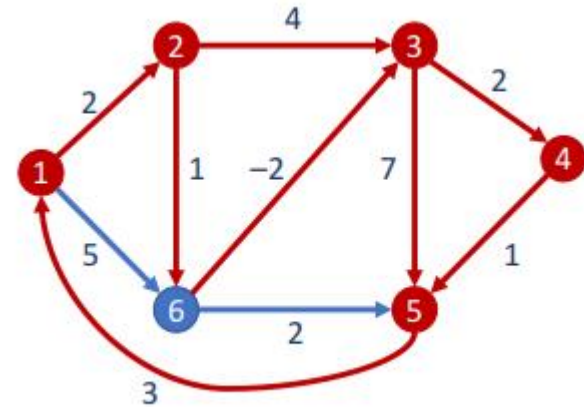
	1	2	3	4	5	6
1	0	2	6	8	9	3
2	∞	0	4	6	7	1
3	∞	∞	0	2	3	∞
4	∞	∞	∞	0	1	∞
5	3	5	9	11	0	6
6	∞	∞	-2	0	1	0

ESEMPIO

```

1:  $W := (w_{ij})$  con  $w_{ij} := \begin{cases} 0 & \text{se } i = j \\ w(i, j) & \text{se } (i, j) \in E(G) \\ \infty & \text{altrimenti} \end{cases}$ 
2:  $D^{(0)} := W$ 
3: per  $k := 1, 2, 3, 4, \boxed{5}$  6 ripeti
4:   per  $i := 1, 2, 3, 4, 5, 6$  ripeti
5:     per  $j := 1, 2, 3, 4, 5, 6$  ripeti
6:        $d_{ij}^{(k)} := \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$ 
7:     fine-ciclo
8:   fine-ciclo
9: fine-ciclo
  
```

- Viene calcolata la matrice $D^{(5)}$ con i costi degli eventuali cammini di costo minimo con vertici intermedi in $\{1, 2, 3, 4, 5\}$



$D^{(5)}$

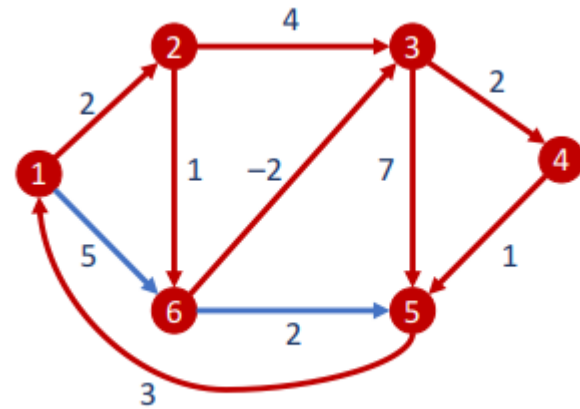
	1	2	3	4	5	6
1	0	2	6	8	9	3
2	10	0	4	6	7	1
3	6	8	0	2	3	9
4	4	6	10	0	1	7
5	3	5	9	11	0	6
6	4	6	-2	0	1	0

ESEMPIO

```

1:  $W := (w_{ij})$  con  $w_{ij} := \begin{cases} 0 & \text{se } i = j \\ w(i, j) & \text{se } (i, j) \in E(G) \\ \infty & \text{altrimenti} \end{cases}$ 
2:  $D^{(0)} := W$ 
3: per  $k := 1, 2, 3, 4, 5, 6$  ripeti
4:   per  $i := 1, 2, 3, 4, 5, 6$  ripeti
5:     per  $j := 1, 2, 3, 4, 5, 6$  ripeti
6:        $d_{ij}^{(k)} := \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$ 
7:     fine-ciclo
8:   fine-ciclo
9: fine-ciclo
  
```

- Viene calcolata la matrice $D^{(6)}$ con i costi degli eventuali cammini di costo minimo con vertici intermedi in $\{1, 2, 3, 4, 5, 6\}$



$D^{(6)}$

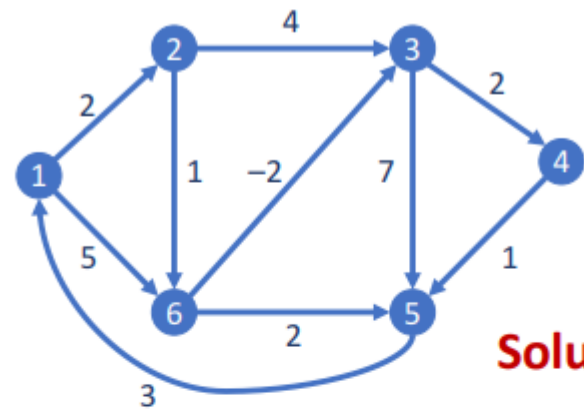
	1	2	3	4	5	6
1	0	2	1	3	4	3
2	6	0	-1	1	2	1
3	6	8	0	2	3	9
4	4	6	5	0	1	7
5	3	5	4	6	0	6
6	4	6	-2	0	1	0

ESEMPIO

```

1:  $W := (w_{ij})$  con  $w_{ij} := \begin{cases} 0 & \text{se } i = j \\ w(i, j) & \text{se } (i, j) \in E(G) \\ \infty & \text{altrimenti} \end{cases}$ 
2:  $D^{(0)} := W$ 
3: per  $k := 1, 2, 3, 4, 5, 6$  ripeti
4:   per  $i := 1, 2, 3, 4, 5, 6$  ripeti
5:     per  $j := 1, 2, 3, 4, 5, 6$  ripeti
6:        $d_{ij}^{(k)} := \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$ 
7:     fine-ciclo
8:   fine-ciclo
9: fine-ciclo
  
```

- Viene calcolata la matrice $D^{(6)}$ con i costi degli eventuali cammini di costo minimo con vertici intermedi in $\{1, 2, 3, 4, 5, 6\}$



Soluzione!

$n = 6$

$D^{(6)}$

	1	2	3	4	5	6
1	0	2	1	3	4	3
2	6	0	-1	1	2	1
3	6	8	0	2	3	9
4	4	6	5	0	1	7
5	3	5	4	6	0	6
6	4	6	-2	0	1	0

Pseudocodice

Algoritmo FloydWarshall (grafo G)

sia $D^0_{xy} = w(x, y)$ se esiste l'arco $x \rightarrow y$, $D^0_{xy} = 0$ se $x = y$, $D^0_{xy} = +\infty$ altrimenti

For $k=1$ to n

For each x, y in V do

$D^k_{xy} = D^{k-1}_{xy}$

if $(D^{k-1}_{xv_k} + D^{k-1}_{v_k y} < D^k_{xy})$ then $D^k_{xy} = D^{k-1}_{xv_k} + D^{k-1}_{v_k y}$

Return D^n

Variante con spazio quadratico

- Lo spazio di memoria cubico diventa proibitivo in molte applicazioni
- Esiste una variante che richiede uno spazio quadratico
- Si può usare una sola matrice anziché una matrice diversa per ogni iterazione k
- Si può osservare che i valori delle celle D^{k-1}_{xvk} e D^{k-1}_{vky} restano gli stessi passando da D^{k-1} a D^k ovvero il passo di rilassamento viene effettuato in termini di valori che non cambiano durante il ciclo for più interno che costruisce D^k a partire da D^{k-1} . Quindi si può usare un'unica matrice D

- Infatti, poiché $D^{k-1}_{vv} = 0$, si ha:

$$D^k_{vk\ v} \min\{D^{k-1}_{xvk}, D^k_{vk} + D^{k-1}_{vk\ v\ k}\} = D^{k-1}_{xvk}$$

$$D^k_{vk\ y} \min\{D^{k-1}_{vky}, D^{k-1}_{vk\ v\ k} + D^{k-1}_{vk\ y}\} = D^{k-1}_{vky}$$

Pseudocodice

Algoritmo FloydWarshall2 (grafo G)

sia $D_{xy}=w(x,y)$ se esiste l'arco $x \rightarrow y$, $D_{xy}=0$ se $x=y$, $D_{xy}=+\infty$ altrimenti

For $k=1$ to n

 For each x,y in V do

 if $(D_{xvk}+D_{vky} < D_{xy})$ then $D_{xy} = D_{xvk}+D_{vky}$

Return D

Una classe per FloydWarshall

```
public class FloydWarshall {
    private boolean hasNegativeCycle;
    // is there a negative cycle?
    private double[][] distTo;
    // distTo[v][w] = length of shortest v->w path
    private DirectedEdge[][] edgeTo;
    // edgeTo[v][w] = last edge on shortest v->w path

    public FloydWarshall(EdgeWeightedDigraph G) {
        int V = G.V();
        distTo = new double[V][V];
        edgeTo = new DirectedEdge[V][V];

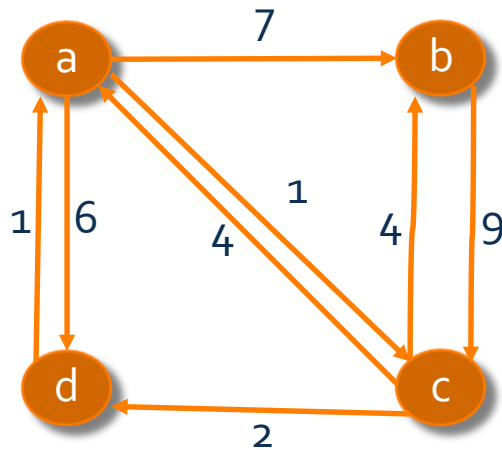
        // initialize distances to infinity
        for (int v = 0; v < V; v++) {
            for (int w = 0; w < V; w++) {
                distTo[v][w] = Double.POSITIVE_INFINITY;
            }
        }

        // initialize distances using edge-weighted digraph's
        for (int v = 0; v < G.V(); v++) {
            for (DirectedEdge e : G.adj(v)) {
                distTo[e.from()][e.to()] = e.weight();
                edgeTo[e.from()][e.to()] = e;
            }
            // in case of self-loops
            if (distTo[v][v] >= 0.0) {
                distTo[v][v] = 0.0;
                edgeTo[v][v] = null;
            }
        }
    }

    // Floyd-Warshall updates
    for (int i = 0; i < V; i++) {
        // compute shortest paths using only 0, 1, ..., i as intermediate
        // vertices
        for (int v = 0; v < V; v++) {
            if (edgeTo[v][i] == null) continue; // optimization
            for (int w = 0; w < V; w++) {
                if (distTo[v][w] > distTo[v][i] + distTo[i][w]) {
                    distTo[v][w] = distTo[v][i] + distTo[i][w];
                    edgeTo[v][w] = edgeTo[i][w];
                }
            }
        }
        // check for negative cycle
        if (distTo[v][v] < 0.0) {
            hasNegativeCycle = true;
            return;
        }
    }
}
```


Esercizio

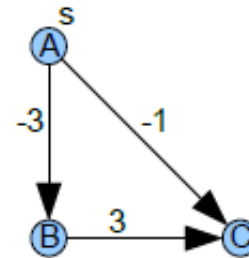
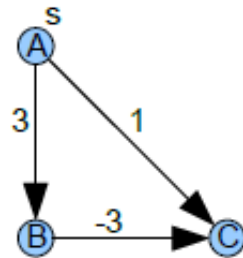
- Mostrare l'esecuzione dell'algoritmo di Floyd-Warshall sul grafo



Soluzione nella cartella materiale aggiuntivo

Esercizi

- Cosa succede se l'algoritmo di Dijkstra è eseguito su un grafo in cui i pesi degli archi possono essere negativi?



Esercizi

- Dati un grafo orientato aciclico e due nodi s ed t , possono esserci molti cammini distinti fra essi. Calcolare il numero dei cammini distinti da s a t . Si utilizzi la programmazione dinamica.

- Soluzione:

```
int pathcount(GRAPH  $G$ , NODE  $s$ , NODE  $t$ )
```

```
  foreach  $v \in V$  do
```

```
     $a[v] \leftarrow -1$ 
```

```
   $a[t] \leftarrow 1$ 
```

```
  return r-pathcount( $G$ ,  $s$ )
```

```
int r-pathcount(GRAPH  $G$ , NODE  $u$ )
```

```
  if  $a[u] < 0$  then
```

```
     $a[u] \leftarrow 0$ 
```

```
    foreach  $v \in G.\text{adj}(u)$  do
```

```
       $a[u] \leftarrow a[u] + \text{r-pathcount}(G, v)$ 
```

```
  return  $a[s]$ 
```



Si utilizza la
programmazione
dinamica

Esercizi: applicazione dei cammini minimi in un grafo?

- implementare Floyd Warshall usando il grafo precedente.
- Dato un insieme di N intervalli della retta reale, si vuole trovare un insieme indipendente massimo, ovvero un insieme X di intervalli avente massima cardinalità, tale che ogni coppia di intervalli abbia intersezione vuota. Si trovi un algoritmo polinomiale.